



Docker: Do Zero ao Avançado

Curso completo com 7 módulos

Conceitos • Imagens • Volumes • Redes • Compose • Segurança •
Produção

CloudTrilhas — cloudtrilhas.com.br

Por Ricardo Simines Scopim



Conceitos Fundamentais e Instalação

Entenda o que é Docker, por que ele revolucionou o desenvolvimento de software, instale no seu sistema e execute seus primeiros containers.

O que é Docker?

Docker é uma plataforma de containerização que permite empacotar aplicações e suas dependências em unidades padronizadas chamadas **containers**. Diferente de máquinas virtuais, containers compartilham o kernel do sistema operacional host, tornando-os extremamente leves e rápidos.

Por que Docker importa?

Consistência

DEV → PROD

"Funciona na minha máquina" deixa de ser problema. O container é idêntico em qualquer ambiente.

Isolamento

SEGURANÇA

Cada container tem seu próprio filesystem, rede e processos isolados do host e de outros containers.

Velocidade

PERFORMANCE

Containers iniciam em segundos (vs minutos de uma VM). Ideal para CI/CD e microserviços.

Portabilidade MULTI-CLOUD

Execute em qualquer lugar: laptop, servidor on-premises, AWS, Azure, GCP — sem alterações.

Conceitos-chave

Conceito	Descrição
Imagem	Template somente-leitura com o sistema de arquivos e configurações. É o "blueprint" do container.
Container	Instância em execução de uma imagem. Possui estado próprio e pode ser iniciado, parado e removido.
Dockerfile	Arquivo de texto com instruções para construir uma imagem customizada passo a passo.
Registry	Repositório de imagens (Docker Hub, ECR, GCR). Permite compartilhar e versionar imagens.
Volume	Mecanismo para persistir dados além do ciclo de vida do container.
Network	Rede virtual que permite comunicação entre containers e com o mundo externo.

Instalação do Docker

Linux (Ubuntu/Debian)

Remover versões antigas LIMPEZA

```
sudo apt-get remove docker docker-engine docker.io containerd runc
```

Instalar dependências

PRÉ-REQUISITOS

```
sudo apt-get update
sudo apt-get install -y \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
```

Adicionar chave GPG oficial

REPOSITÓRIO

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

Adicionar repositório Docker

APT

```
echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(lsb_release -cs) stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Instalar Docker Engine

INSTALAÇÃO

```
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
```

Pós-instalação (sem sudo)

PERMISSÕES

```
sudo usermod -aG docker $USER  
newgrp docker
```

Windows

Baixe e instale o **Docker Desktop for Windows** em docker.com. Requisitos: Windows 10/11 64-bit com WSL 2 habilitado ou Hyper-V.

macOS

Baixe e instale o **Docker Desktop for Mac** em docker.com. Disponível para chips Intel e Apple Silicon (M1/M2/M3).

Verificação da Instalação

docker --version

VERSÃO

Exibe a versão instalada do Docker CLI.

```
docker --version  
# Docker version 24.0.7, build afdd53b
```

docker info

DETALHES

Mostra informações detalhadas do Docker Engine: storage driver, runtime, plugins, etc.

```
docker info
```

docker run hello-world TESTE

Baixa e executa a imagem de teste oficial. Se funcionar, sua instalação está correta.

```
docker run hello-world
```

Primeiros Comandos Essenciais

docker run — Criar e executar containers

docker run BÁSICO

Cria e inicia um container a partir de uma imagem.

```
# Execução simples
docker run nginx

# Modo interativo com terminal
docker run -it ubuntu bash

# Modo detached (background)
docker run -d nginx

# Mapear porta host:container
docker run -d -p 8080:80 nginx

# Nomear o container
docker run -d --name meu-nginx -p 8080:80 nginx

# Remover automaticamente ao parar
docker run --rm -it alpine sh
```

Flags importantes

REFERÊNCIA

Flag	Descrição
<code>-it</code>	Interativo + pseudo-TTY (terminal)
<code>-d</code>	Detached — executa em background
<code>-p host:container</code>	Mapeia porta do host para o container
<code>--name</code>	Define um nome para o container
<code>--rm</code>	Remove o container ao finalizar
<code>-e VAR=valor</code>	Define variável de ambiente
<code>-v host:container</code>	Monta volume ou bind mount
<code>--network</code>	Conecta a uma rede específica

docker ps — Listar containers

docker ps

LISTAR

Lista containers em execução. Use flags para ver todos ou filtrar.

```
# Containers em execução
docker ps

# Todos os containers (incluindo parados)
docker ps -a

# Apenas IDs dos containers
docker ps -q

# Todos os IDs (para scripts)
docker ps -aq
```

docker stop / start / restart — Controlar containers

docker stop PARAR

Envia SIGTERM e depois SIGKILL (após timeout) para parar o container graciosamente.

```
# Parar um container
docker stop meu-nginx

# Parar múltiplos containers
docker stop container1 container2

# Parar todos os containers em execução
docker stop $(docker ps -q)
```

docker start / restart INICIAR

Inicia um container parado ou reinicia um container em execução.

```
# Iniciar container parado
docker start meu-nginx

# Reiniciar container
docker restart meu-nginx
```

docker rm — Remover containers

docker rm REMOVER

Remove containers parados. Use `-f` para forçar remoção de containers em execução.

```
# Remover container parado
docker rm meu-nginx

# Forçar remoção (mesmo em execução)
docker rm -f meu-nginx

# Remover todos os containers parados
docker container prune

# Remover todos os containers (forçado)
docker rm -f $(docker ps -aq)
```

docker logs — Ver logs do container

docker logs LOGS

Exibe a saída (stdout/stderr) de um container. Essencial para debugging.

```
# Ver todos os logs
docker logs meu-nginx

# Seguir logs em tempo real (like tail -f)
docker logs -f meu-nginx

# Últimas N linhas
docker logs --tail 50 meu-nginx

# Com timestamps
docker logs -t meu-nginx

# Combinar flags
docker logs -f --tail 100 -t meu-nginx
```

docker exec — Executar comandos em container ativo

docker exec EXECUTAR

Executa um comando dentro de um container em execução. Ideal para debugging e administração.

```
# Abrir shell interativo
docker exec -it meu-nginx bash

# Executar comando único
docker exec meu-nginx cat /etc/nginx/nginx.conf

# Executar como root
docker exec -u root meu-nginx apt-get update

# Definir variável de ambiente
docker exec -e MY_VAR=teste meu-nginx env
```

docker inspect — Detalhes do container

docker inspect INSPECIONAR

Retorna informações detalhadas em JSON sobre um container, imagem, volume ou rede.

```
# Inspecionar container (JSON completo)
docker inspect meu-nginx

# Obter IP do container
docker inspect --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' meu-nginx

# Ver mounts/volumes
docker inspect --format '{{json .Mounts}}' meu-nginx

# Ver variáveis de ambiente
docker inspect --format '{{json .Config.Env}}' meu-nginx

# Ver status
docker inspect --format '{{.State.Status}}' meu-nginx
```



Imagens e Dockerfile

Domine o gerenciamento de imagens Docker e aprenda a construir imagens customizadas com Dockerfile — a base para containerizar qualquer aplicação.

O que é uma Imagem Docker?

Uma imagem Docker é um pacote somente-leitura composto por **camadas (layers)**. Cada instrução no Dockerfile cria uma nova camada. Camadas são compartilhadas entre imagens, economizando espaço e acelerando downloads.

Cache de camadas: O Docker reutiliza camadas que não mudaram. Se você alterar apenas a última instrução do Dockerfile, todas as camadas anteriores vêm do cache — tornando builds subsequentes muito mais rápidos.

Comandos para Gerenciar Imagens

docker images

LISTAR

Lista todas as imagens locais com tamanho, tag e ID.

```
# Listar imagens
docker images

# Filtrar por nome
docker images nginx

# Mostrar apenas IDs
docker images -q
```

docker pull **BAIXAR**

Baixa uma imagem do registry (Docker Hub por padrão).

```
# Baixar última versão
docker pull nginx

# Baixar versão específica
docker pull nginx:1.25-alpine

# Baixar de registry privado
docker pull meu-registry.com/app:v2.0
```

docker rmi **REMOVER**

Remove imagens locais. Falha se houver containers usando a imagem.

```
# Remover imagem
docker rmi nginx:1.25-alpine

# Forçar remoção
docker rmi -f nginx

# Remover imagens sem tag (dangling)
docker image prune

# Remover TODAS as imagens não utilizadas
docker image prune -a
```

docker tag TAGUEAR

Cria uma nova tag (alias) para uma imagem existente.

```
# Taguear para push
docker tag minha-app:latest meu-user/minha-app:v1.0

# Taguear para registry privado
docker tag minha-app:latest 123456789.dkr.ecr.us-east-1.amazonaws.com/minha-ap
```

docker push ENVIAR

Envia uma imagem local para um registry remoto.

```
# Login no Docker Hub
docker login

# Push para Docker Hub
docker push meu-user/minha-app:v1.0

# Push para ECR (AWS)
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/minha-app:v1.0
```

docker history HISTÓRICO

Mostra as camadas de uma imagem e os comandos que as criaram.

```
# Ver camadas da imagem
docker history nginx

# Sem truncar comandos
docker history --no-trunc nginx
```

Estrutura do Dockerfile

O Dockerfile é um arquivo de texto com instruções sequenciais para construir uma imagem. Cada instrução cria uma camada.

FROM OBRIGATÓRIO

Define a imagem base. Sempre é a primeira instrução.

```
FROM node:20-alpine
FROM python:3.12-slim
FROM ubuntu:22.04
FROM scratch # imagem vazia
```

WORKDIR DIRETÓRIO

Define o diretório de trabalho para instruções subsequentes (RUN, CMD, COPY, etc.).

```
WORKDIR /app
# Cria o diretório se não existir
```

COPY COPIAR

Copia arquivos/diretórios do contexto de build para a imagem.

```
# Copiar arquivo específico
COPY package.json .

# Copiar tudo
COPY . .

# Copiar com permissões
COPY --chown=node:node . .
```

ADD **ADICIONAR**

Similar ao COPY, mas também extrai arquivos .tar e aceita URLs. Prefira COPY na maioria dos casos.

```
# Extrair tar automaticamente
ADD app.tar.gz /app/

# Baixar de URL (evite - prefira curl no RUN)
ADD https://example.com/file.txt /app/
```

RUN **EXECUTAR**

Executa comandos durante o build. Cada RUN cria uma nova camada.

```
# Instalar dependências
RUN apt-get update && apt-get install -y \
    curl \
    vim \
    && rm -rf /var/lib/apt/lists/*

# Instalar pacotes Node.js
RUN npm ci --only=production
```

CMD **COMANDO PADRÃO**

Define o comando padrão ao iniciar o container. Pode ser sobrescrito no `docker run`.

```
# Forma exec (recomendada)
CMD ["node", "server.js"]

# Forma shell
CMD npm start
```


ENTRYPOINT PONTO DE ENTRADA

Define o executável principal do container. Não é facilmente sobrescrito.

```
# Container como executável
ENTRYPOINT ["python", "app.py"]

# Combinado com CMD (args padrão)
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8000"]
```

EXPOSE PORTA

Documenta a porta que o container escuta. Não publica a porta — é apenas metadata.

```
EXPOSE 3000
EXPOSE 8080/tcp
EXPOSE 8443/udp
```

ENV VARIÁVEL

Define variáveis de ambiente disponíveis no build E no container em execução.

```
ENV NODE_ENV=production
ENV PORT=3000
ENV APP_HOME=/app
```

ARG BUILD ARG

Define variáveis disponíveis apenas durante o build. Não persiste no container final.

```
ARG NODE_VERSION=20
FROM node:${NODE_VERSION}-alpine

ARG BUILD_DATE
LABEL build-date=$BUILD_DATE
```

VOLUME VOLUME

Cria um ponto de montagem para dados persistentes.

```
VOLUME /data
VOLUME ["/var/log", "/data"]
```

USER SEGURANÇA

Define o usuário para executar instruções subsequentes e o container. Evite rodar como root.

```
# Criar e usar usuário não-root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
```

LABEL **METADATA**

Adiciona metadados à imagem (autor, versão, descrição).

```
LABEL maintainer="dev@example.com"
LABEL version="1.0"
LABEL description="API de pagamentos"
```

HEALTHCHECK **SAÚDE**

Define como verificar se o container está saudável.

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1
```

ENTRYPOINT vs CMD

Aspecto	CMD	ENTRYPOINT
Sobrescrito por	Argumentos no <code>docker run</code>	Flag <code>--entrypoint</code>
Uso típico	Argumentos padrão	Executável principal
Combinação	ENTRYPOINT define o comando, CMD define argumentos padrão	

Exemplo combinado **PADRÃO**

```
ENTRYPOINT ["python", "manage.py"]
CMD ["runserver", "0.0.0.0:8000"]

# docker run app → python manage.py runserver 0.0.0.0:8000
# docker run app migrate → python manage.py migrate
```

ARG vs ENV

Aspecto	ARG	ENV
Disponível no build	✓ Sim	✓ Sim
Disponível no container	✗ Não	✓ Sim
Definido via CLI	<code>--build-arg</code>	<code>-e</code> no run
Persiste na imagem	✗ Não	✓ Sim

HEALTHCHECK — Exemplo Completo

HEALTHCHECK

MONITORAMENTO

O Docker verifica periodicamente a saúde do container. Status: `healthy`, `unhealthy`, `starting`.

```
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm ci --only=production
EXPOSE 3000

HEALTHCHECK --interval=30s \
  --timeout=10s \
  --start-period=10s \
  --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

CMD ["node", "server.js"]
```

USER — Segurança com Usuário Não-Root

USER**SEGURANÇA**

Nunca execute containers como root em produção. Crie um usuário dedicado.

```
FROM node:20-alpine

# Criar usuário não-root
RUN addgroup -S appgroup && \
    adduser -S appuser -G appgroup

WORKDIR /app
COPY --chown=appuser:appgroup . .
RUN npm ci --only=production

# Trocar para usuário não-root
USER appuser

EXPOSE 3000
CMD ["node", "server.js"]
```

docker build — Construir Imagens

docker build **BUILD**

Constrói uma imagem a partir de um Dockerfile e um contexto de build.

```
# Build básico (Dockerfile no diretório atual)
docker build -t minha-app:v1.0 .

# Especificar Dockerfile diferente
docker build -f Dockerfile.prod -t minha-app:prod .

# Passar build arguments
docker build --build-arg NODE_VERSION=20 -t minha-app .

# Build sem cache (forçar rebuild completo)
docker build --no-cache -t minha-app .

# Ver output detalhado do build
docker build --progress=plain -t minha-app .

# Múltiplas tags
docker build -t minha-app:latest -t minha-app:v1.0 .
```

.dockerignore

.dockerignore **OTIMIZAÇÃO**

Exclui arquivos do contexto de build. Reduz tamanho, acelera build e evita copiar dados sensíveis.

```
# Dependências (serão instaladas no build)
node_modules/
__pycache__/
venv/

# Controle de versão
.git/
.gitignore

# IDE e editor
.vscode/
.idea/
*.swp

# Ambiente e segredos
.env
.env.*
*.pem
*.key

# Build artifacts
dist/
build/
*.log

# Docker
Dockerfile*
docker-compose*
.dockerignore
```

Exemplo Completo — Dockerfile Node.js

Dockerfile

NODE.JS


```
# Imagem base leve
FROM node:20-alpine

# Metadados
LABEL maintainer="dev@example.com"
LABEL version="1.0"

# Criar usuário não-root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Diretório de trabalho
WORKDIR /app

# Copiar apenas package files primeiro (cache de dependências)
COPY package.json package-lock.json ./

# Instalar dependências
RUN npm ci --only=production && npm cache clean --force

# Copiar código da aplicação
COPY --chown=appuser:appgroup . .

# Variáveis de ambiente
ENV NODE_ENV=production
ENV PORT=3000

# Trocar para usuário não-root
USER appuser

# Documentar porta
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

# Comando de inicialização
CMD ["node", "server.js"]
```

Exemplo Completo — Dockerfile Python

Dockerfile

PYTHON

```
# Imagem base slim
FROM python:3.12-slim

# Metadados
LABEL maintainer="dev@example.com"

# Evitar criação de .pyc e buffering
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Instalar dependências do sistema
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Criar usuário não-root
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

# Diretório de trabalho
WORKDIR /app

# Copiar e instalar dependências Python
COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Copiar código
COPY --chown=appuser:appgroup . .

# Trocar para usuário não-root
USER appuser

# Porta
EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

# Iniciar aplicação
```

```
ENTRYPOINT ["python"]
```

```
CMD ["app.py", "--host", "0.0.0.0", "--port", "8000"]
```



Volumes e Persistência de Dados

Containers são efêmeros por natureza. Aprenda a persistir dados com volumes, bind mounts e tmpfs — e domine backup, restauração e transferência de dados.

O Problema: Containers são Efêmeros

Quando um container é removido, todos os dados escritos dentro dele são perdidos. Isso é por design — containers devem ser descartáveis e recriáveis. Mas aplicações reais precisam persistir dados: bancos de dados, uploads de usuários, logs, configurações.

Regra de ouro: Nunca armazene dados importantes apenas dentro do filesystem do container. Use volumes ou bind mounts para qualquer dado que precisa sobreviver à remoção do container.

Tipos de Armazenamento

Tipo	Localização	Gerenciado por	Uso Ideal
Volume	<code>/var/lib/docker/volumes/</code>	Docker	Dados de produção, bancos de dados, dados compartilhados entre containers
Bind Mount	Qualquer caminho no host	Usuário	Desenvolvimento (código-fonte), configurações do host
tmpfs	Memória RAM do host	Kernel	Dados temporários sensíveis, cache em memória

Comandos para Gerenciar Volumes

`docker volume create` CRIAR

Cria um volume nomeado gerenciado pelo Docker.

```
# Criar volume
docker volume create meu-volume

# Criar com driver específico
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=192.168.1.100,rw \
  --opt device=:/path/to/dir \
  nfs-volume
```

`docker volume ls` LISTAR

Lista todos os volumes existentes no host.

```
# Listar volumes
docker volume ls

# Filtrar por driver
docker volume ls --filter driver=local

# Filtrar volumes não utilizados (dangling)
docker volume ls --filter dangling=true
```

docker volume inspect INSPECIONAR

Mostra detalhes do volume: driver, mountpoint, labels, etc.

```
docker volume inspect meu-volume

# Saída:
# [{ "CreatedAt": "...",
#   "Driver": "local",
#   "Mountpoint": "/var/lib/docker/volumes/meu-volume/_data",
#   "Name": "meu-volume" }]
```

docker volume rm REMOVER

Remove um ou mais volumes. Falha se o volume estiver em uso.

```
# Remover volume específico
docker volume rm meu-volume

# Remover múltiplos
docker volume rm vol1 vol2 vol3
```

docker volume prune LIMPAR

Remove TODOS os volumes não utilizados por nenhum container. Use com cuidado!

```
# Remover volumes órfãos
docker volume prune

# Sem confirmação
docker volume prune -f
```

Usando Volumes com Containers

-v (volume) SINTAXE CURTA

Monta um volume nomeado no container. Cria o volume automaticamente se não existir.

```
# Montar volume nomeado
docker run -d -v meu-volume:/app/data nginx

# Volume anônimo (Docker gera nome aleatório)
docker run -d -v /app/data nginx

# Volume somente-leitura
docker run -d -v meu-volume:/app/data:ro nginx
```

--mount (volume) SINTAXE EXPLÍCITA

Sintaxe mais verbosa e clara. Recomendada para scripts e documentação.

```
# Montar volume nomeado
docker run -d \
  --mount type=volume,source=meu-volume,target=/app/data \
  nginx

# Volume somente-leitura
docker run -d \
  --mount type=volume,source=meu-volume,target=/app/data,readonly \
  nginx
```

Demonstração de Persistência — MySQL

MySQL com Volume

EXEMPLO PRÁTICO

Dados do MySQL persistem mesmo após remover e recriar o container.

```
# Criar volume para dados do MySQL
docker volume create mysql-data

# Iniciar MySQL com volume
docker run -d \
  --name meu-mysql \
  -e MYSQL_ROOT_PASSWORD=senha123 \
  -e MYSQL_DATABASE=app_db \
  -v mysql-data:/var/lib/mysql \
  -p 3306:3306 \
  mysql:8.0

# Criar dados no banco
docker exec -it meu-mysql mysql -uroot -psenha123 -e \
  "CREATE TABLE app_db.users (id INT, name VARCHAR(50)); \
  INSERT INTO app_db.users VALUES (1, 'João');"

# Remover o container
docker rm -f meu-mysql

# Recriar — dados ainda estão lá!
docker run -d \
  --name meu-mysql \
  -e MYSQL_ROOT_PASSWORD=senha123 \
  -v mysql-data:/var/lib/mysql \
  -p 3306:3306 \
  mysql:8.0

# Verificar dados
docker exec -it meu-mysql mysql -uroot -psenha123 -e \
  "SELECT * FROM app_db.users;"

# → 1 | João ✓
```

Bind Mounts

Bind mounts mapeiam um diretório do host diretamente no container. Ideal para desenvolvimento — alterações no código são refletidas instantaneamente.

Linux / macOS

BIND MOUNT

```
# Montar diretório atual
docker run -d -v $(pwd):/app -p 3000:3000 node:20-alpine

# Caminho absoluto
docker run -d -v /home/user/projeto:/app nginx

# Somente-leitura
docker run -d -v $(pwd)/config:/etc/nginx/conf.d:ro nginx
```

Windows PowerShell

BIND MOUNT

```
# Montar diretório atual (PowerShell)
docker run -d -v ${PWD}:/app -p 3000:3000 node:20-alpine

# Caminho absoluto
docker run -d -v C:\Users\dev\projeto:/app nginx
```

Windows CMD

BIND MOUNT

```
# Montar diretório atual (CMD)
docker run -d -v %cd%:/app -p 3000:3000 node:20-alpine
```

--mount (bind) SINTAXE EXPLÍCITA

Sintaxe `--mount` para bind mounts. Falha se o diretório source não existir (mais seguro).

```
docker run -d \  
  --mount type=bind,source=$(pwd)/src,target=/app/src \  
  --mount type=bind,source=$(pwd)/config,target=/app/config,readonly \  
  -p 3000:3000 \  
  node:20-alpine
```

tmpfs — Armazenamento em Memória

--tmpfs MEMÓRIA

Monta um filesystem temporário em RAM. Dados são perdidos ao parar o container. Ideal para dados sensíveis temporários.

```
# tmpfs simples  
docker run -d --tmpfs /app/temp nginx  
  
# Com --mount (mais opções)  
docker run -d \  
  --mount type=tmpfs,target=/app/temp,tmpfs-size=100m,tmpfs-mode=1770 \  
  nginx  
  
# Uso: cache de sessão, tokens temporários, dados sensíveis
```

Backup e Restauração de Volumes

Backup TAR

Cria um arquivo tar com o conteúdo do volume usando um container auxiliar.

```
# Backup de volume para arquivo tar
docker run --rm \
  -v meu-volume:/data \
  -v $(pwd):/backup \
  alpine tar czf /backup/meu-volume-backup.tar.gz -C /data .

# Explicação:
# - Monta o volume em /data (leitura)
# - Monta diretório atual em /backup (escrita)
# - Cria tar.gz com conteúdo de /data
```

Restore TAR

Restaura dados de um arquivo tar para um volume.

```
# Criar volume de destino
docker volume create meu-volume-restaurado

# Restaurar backup
docker run --rm \
  -v meu-volume-restaurado:/data \
  -v $(pwd):/backup \
  alpine tar xzf /backup/meu-volume-backup.tar.gz -C /data

# Verificar
docker run --rm -v meu-volume-restaurado:/data alpine ls -la /data
```

docker cp — Copiar Arquivos

docker cp COPIAR

Copia arquivos entre o host e um container (em execução ou parado). Útil para debugging e extração de dados.

```
# Host → Container
docker cp ./config.json meu-container:/app/config.json
docker cp ./src/ meu-container:/app/src/

# Container → Host
docker cp meu-container:/app/logs/error.log ./error.log
docker cp meu-container:/var/log/ ./logs-extraídos/

# Funciona com container parado também
docker cp container-parado:/data/dump.sql ./backup.sql
```



Redes no Docker

Entenda como containers se comunicam entre si e com o mundo externo. Domine drivers de rede, DNS interno, port mapping e crie arquiteturas multi-container robustas.

Por que Redes Docker Importam?

Em aplicações reais, containers precisam se comunicar: uma API precisa acessar o banco de dados, um frontend precisa chamar o backend, um cache Redis precisa ser acessível por múltiplos serviços. O Docker fornece um sistema de rede completo com isolamento, DNS automático e diferentes drivers para diferentes cenários.

Drivers de Rede

Driver	Descrição	Uso Ideal
bridge	Rede padrão. Cria uma bridge virtual no host. Containers na mesma bridge se comunicam.	Comunicação entre containers no mesmo host
host	Remove isolamento de rede. Container usa a rede do host diretamente.	Performance máxima, sem overhead de NAT
none	Desabilita toda conectividade de rede.	Containers que não precisam de rede (batch jobs isolados)
overlay	Conecta containers em múltiplos hosts Docker (Swarm/Kubernetes).	Clusters multi-host, Docker Swarm
macvlan	Atribui um endereço MAC ao container, fazendo-o aparecer como dispositivo físico na rede.	Integração com rede física, aplicações legadas

Comandos para Gerenciar Redes

docker network ls LISTAR

Lista todas as redes Docker disponíveis.

```
docker network ls

# Saída padrão:
# NETWORK ID      NAME           DRIVER        SCOPE
# a1b2c3d4e5f6    bridge        bridge        local
# f6e5d4c3b2a1    host          host          local
# 1a2b3c4d5e6f    none          null          local
```

docker network create CRIAR

Cria uma rede customizada. Redes customizadas têm DNS automático entre containers.

```
# Criar rede bridge customizada
docker network create minha-rede

# Criar com subnet específica
docker network create \
  --driver bridge \
  --subnet 172.20.0.0/16 \
  --gateway 172.20.0.1 \
  minha-rede

# Criar rede overlay (Swarm)
docker network create --driver overlay minha-overlay
```


docker network inspect INSPECIONAR

Mostra detalhes da rede: subnet, gateway, containers conectados, etc.

```
docker network inspect minha-rede

# Ver apenas containers conectados
docker network inspect minha-rede --format '{{json .Containers}}'
```

docker network rm REMOVER

Remove uma rede. Falha se houver containers conectados.

```
# Remover rede
docker network rm minha-rede

# Remover múltiplas
docker network rm rede1 rede2
```

docker network connect CONECTAR

Conecta um container em execução a uma rede adicional.

```
# Conectar container a rede
docker network connect minha-rede meu-container

# Conectar com IP fixo
docker network connect --ip 172.20.0.10 minha-rede meu-container
```

docker network disconnect DESCONECTAR

Remove um container de uma rede.

```
# Desconectar container
docker network disconnect minha-rede meu-container

# Forçar desconexão
docker network disconnect -f minha-rede meu-container
```

docker network prune LIMPAR

Remove todas as redes não utilizadas.

```
docker network prune

# Sem confirmação
docker network prune -f
```

Comunicação entre Containers

Bridge Padrão vs Bridge Customizada

Bridge padrão SEM DNS

Na rede bridge padrão, containers só se comunicam por IP. Não há resolução DNS por nome.

```
# Containers na bridge padrão
docker run -d --name app1 nginx
docker run -d --name app2 nginx

# Comunicação apenas por IP (não por nome)
docker exec app1 ping app2 #  FALHA

# Precisa descobrir o IP manualmente
docker inspect --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' app1
# → 172.17.0.3
docker exec app1 ping 172.17.0.3 #  Funciona
```

Bridge customizada DNS AUTOMÁTICO

Em redes customizadas, o Docker fornece DNS automático. Containers se encontram pelo nome.

```
# Criar rede customizada
docker network create app-network

# Containers na rede customizada
docker run -d --name app1 --network app-network nginx
docker run -d --name app2 --network app-network nginx

# Comunicação por nome 
docker exec app1 ping app2 #  Funciona!
docker exec app2 ping app1 #  Funciona!

# DNS resolve automaticamente
docker exec app1 nslookup app2
# → Address: 172.18.0.3
```

Exemplo Prático — App + Redis + PostgreSQL

Arquitetura multi-container

EXEMPLO COMPLETO

Três containers se comunicando via DNS na mesma rede customizada.

```
# 1. Criar rede da aplicação
docker network create app-stack

# 2. Iniciar PostgreSQL
docker run -d \
  --name postgres \
  --network app-stack \
  -e POSTGRES_USER=app \
  -e POSTGRES_PASSWORD=secret \
  -e POSTGRES_DB=myapp \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16-alpine

# 3. Iniciar Redis
docker run -d \
  --name redis \
  --network app-stack \
  redis:7-alpine

# 4. Iniciar aplicação (conecta ao DB e cache por nome)
docker run -d \
  --name api \
  --network app-stack \
  -e DATABASE_URL=postgresql://app:secret@postgres:5432/myapp \
  -e REDIS_URL=redis://redis:6379 \
  -p 3000:3000 \
  minha-api:latest

# 5. Verificar conectividade
docker exec api ping postgres # ✓
docker exec api ping redis    # ✓

# 6. Ver containers na rede
docker network inspect app-stack --format '{{range .Containers}}{{.Name}} {{end}}'
# → postgres redis api
```

Rede Host

--network host SEM ISOLAMENTO

O container compartilha a rede do host. Sem NAT, sem port mapping — a porta do container é a porta do host diretamente. Apenas Linux.

```
# Container usa rede do host diretamente
docker run -d --network host nginx

# Nginx agora escuta na porta 80 do HOST
# Não precisa de -p (port mapping)
curl http://localhost:80 # ✓ Acessa o nginx

# Útil para: monitoramento, performance máxima
# Desvantagem: sem isolamento de rede
```

Rede None

--network none ISOLADO

Container completamente isolado da rede. Apenas interface loopback disponível.

```
# Container sem rede
docker run -d --network none alpine sleep 3600

# Verificar — apenas loopback
docker exec -it $(docker ps -q -l) ip addr
# → Apenas lo (127.0.0.1)

# Uso: processamento batch, tarefas que não precisam de rede
```

Port Mapping (Mapeamento de Portas)

-p (publish) **PORTAS**

Mapeia portas do host para portas do container, permitindo acesso externo.

```
# host:container
docker run -d -p 8080:80 nginx

# Múltiplas portas
docker run -d -p 8080:80 -p 8443:443 nginx

# Porta aleatória do host
docker run -d -p 80 nginx

# Bind em interface específica
docker run -d -p 127.0.0.1:8080:80 nginx

# Bind em interface específica com porta aleatória
docker run -d -p 127.0.0.1::80 nginx

# Range de portas
docker run -d -p 8000-8010:8000-8010 minha-app

# UDP
docker run -d -p 5353:53/udp dns-server
```

docker port [VERIFICAR](#)

Mostra o mapeamento de portas de um container.

```
# Ver portas mapeadas
docker port meu-nginx

# Saída:
# 80/tcp -> 0.0.0.0:8080
# 443/tcp -> 0.0.0.0:8443

# Porta específica
docker port meu-nginx 80
# -> 0.0.0.0:8080
```

DNS Aliases (--network-alias)

--network-alias DNS

Permite que múltiplos containers respondam pelo mesmo nome DNS. Útil para load balancing básico e service discovery.

```
# Criar rede
docker network create app-net

# Dois containers com mesmo alias
docker run -d \
  --name web1 \
  --network app-net \
  --network-alias web \
  nginx

docker run -d \
  --name web2 \
  --network app-net \
  --network-alias web \
  nginx

# Resolver alias — retorna IPs de ambos (round-robin DNS)
docker run --rm --network app-net alpine nslookup web
# → Name: web  Address: 172.18.0.2
# → Name: web  Address: 172.18.0.3

# Requisições para "web" são distribuídas entre web1 e web2
docker run --rm --network app-net alpine wget -qO- http://web
```



Docker — Módulo 05

Docker Compose

Orquestre múltiplos containers de forma declarativa, reproduzível e versionável. Defina toda a infraestrutura da sua aplicação em um único arquivo YAML.

O que é Docker Compose e Por que Usar

Orquestração Local

CONCEITO

Docker Compose é uma ferramenta para definir e executar aplicações multi-container. Com um único arquivo YAML você descreve todos os serviços, redes e volumes da sua aplicação.

```
# Vantagens principais:  
# ✓ Declarativo – infraestrutura como código  
# ✓ Reproduzível – mesmo resultado em qualquer máquina  
# ✓ Versionável – arquivo YAML no Git  
# ✓ Um comando – sobe toda a stack  
# ✓ Isolamento – cada projeto em sua rede
```

Estrutura do docker-compose.yml

docker-compose.yml**ESTRUTURA**

Anatomia completa de um arquivo Compose com todas as diretivas principais.

```

services:
  nome-servico:
    image: imagem:tag          # Imagem do Docker Hub ou registry
    build:                     # OU build a partir de Dockerfile
      context: ./dir
      dockerfile: Dockerfile
    args:
      NODE_ENV: production
    container_name: meu-app    # Nome fixo do container
    ports:
      - "8080:80"              # host:container
      - "443:443"
    volumes:
      - ./src:/app/src          # Bind mount (dev)
      - dados:/var/lib/data     # Named volume (persistência)
    environment:
      - DB_HOST=postgres
      - DB_PORT=5432
    env_file:
      - .env                    # Variáveis de arquivo externo
    depends_on:
      - db
      - cache
    networks:
      - frontend
      - backend
    restart: unless-stopped    # Política de reinício

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge

volumes:
  dados:
    driver: local

```

Exemplo Completo: Web + API + DB + Cache

`docker-compose.yml`**EXEMPLO**

Stack completa com frontend React, API Node.js, PostgreSQL e Redis.

```
services:
  web:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - api
    networks:
      - frontend

  api:
    build: ./backend
    ports:
      - "4000:4000"
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/app
      - REDIS_URL=redis://cache:6379
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
    networks:
      - frontend
      - backend
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: app
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 10s
```

```
    timeout: 5s
    retries: 5
  networks:
    - backend

cache:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  volumes:
    - redisdata:/data
  networks:
    - backend

networks:
  frontend:
  backend:

volumes:
  pgdata:
  redisdata:
```

Comandos Essenciais

docker compose up INICIAR

Cria e inicia todos os containers definidos no compose.

```
# Modo detached (background)
docker compose up -d

# Forçar rebuild das imagens
docker compose up --build

# Rebuild + detached
docker compose up -d --build

# Subir serviço específico
docker compose up -d api
```

docker compose down PARAR

Para e remove containers, redes criadas pelo compose.

```
# Parar e remover containers + redes
docker compose down

# Remover também os volumes
docker compose down -v

# Remover imagens locais criadas
docker compose down --rmi local

# Remover TODAS as imagens usadas
docker compose down --rmi all
```

docker compose ps STATUS

Lista o status de todos os containers do projeto.

```
# Listar containers do projeto
docker compose ps

# Formato compacto
docker compose ps --format "table {{.Name}}\t{{.Status}}"

# Incluir containers parados
docker compose ps -a
```

docker compose logs DEBUG

Visualiza logs dos containers do projeto.

```
# Logs de todos os serviços
docker compose logs

# Follow (tempo real)
docker compose logs -f

# Logs de serviço específico
docker compose logs -f api

# Últimas 50 linhas
docker compose logs --tail 50 api
```

docker compose stop/start/restart

CONTROLE

Controla o ciclo de vida sem remover containers.

```
# Parar sem remover
docker compose stop

# Iniciar containers parados
docker compose start

# Reiniciar todos
docker compose restart

# Reiniciar serviço específico
docker compose restart api
```

docker compose exec

EXECUTAR

Executa comando em container já em execução.

```
# Shell interativo
docker compose exec api sh

# Comando direto
docker compose exec db psql -U user -d app

# Como root
docker compose exec -u root api sh
```

docker compose run EXECUTAR

Cria container temporário para executar comando único.

```
# Executar e remover ao final
docker compose run --rm api npm test

# Executar migrations
docker compose run --rm api npx prisma migrate deploy

# Sem iniciar dependências
docker compose run --rm --no-deps api npm run lint
```

docker compose scale / build / pull GERENCIAR

Escalar réplicas, buildar imagens e baixar atualizações.

```
# Escalar serviço (múltiplas réplicas)
docker compose up -d --scale api=3

# Build sem iniciar
docker compose build

# Build sem cache
docker compose build --no-cache

# Baixar imagens atualizadas
docker compose pull
```

Recursos Avançados

depends_on + healthcheck**DEPENDÊNCIA**

Garanta que serviços só iniciem quando dependências estiverem saudáveis.

```
services:
  api:
    depends_on:
      db:
        condition: service_healthy
    cache:
      condition: service_started

  db:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
```

.env + Variáveis CONFIG

Use arquivos .env para substituição de variáveis no compose.

```
# .env
POSTGRES_VERSION=16-alpine
APP_PORT=4000
DB_PASSWORD=supersecret

# docker-compose.yml
services:
  db:
    image: postgres:${POSTGRES_VERSION}
  api:
    ports:
      - "${APP_PORT}:4000"
    environment:
      - DB_PASS=${DB_PASSWORD}
```

Profiles

AVANÇADO

Agrupe serviços em perfis para ativar sob demanda.

```
services:
  web:
    build: ./frontend
    # Sempre ativo (sem profile)

  debug:
    image: busybox
    profiles: ["debug"]

  monitoring:
    image: grafana/grafana
    profiles: ["monitoring"]

# Uso:
# docker compose up -d
# docker compose --profile debug up -d
# docker compose --profile monitoring up -d
```

Múltiplos Compose Files

AVANÇADO

Combine arquivos para diferentes ambientes (override pattern).

```
# docker-compose.yml (base)
# docker-compose.override.yml (dev - auto-carregado)
# docker-compose.prod.yml (produção)

# Dev (carrega base + override automaticamente)
docker compose up -d

# Produção (especificar arquivos)
docker compose -f docker-compose.yml \
  -f docker-compose.prod.yml up -d

# Override desabilita volumes de dev
# e usa imagens do registry
```

Restart Policies

RESILIÊNCIA

Defina como containers devem se comportar após falhas.

```
services:
  api:
    restart: "no"          # Nunca reinicia (padrão)

  web:
    restart: always        # Sempre reinicia

  worker:
    restart: on-failure     # Só se exit code != 0

  db:
    restart: unless-stopped # Sempre, exceto se
                           # parado manualmente
```


Resource Limits

RECURSOS

Limite CPU e memória por serviço para evitar consumo excessivo.

```
services:
  api:
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
        reservations:
          cpus: "0.25"
          memory: 128M

  worker:
    deploy:
      resources:
        limits:
          cpus: "2.0"
          memory: 1G
```

Resumo de Comandos

Comando	Descrição
<code>docker compose up -d</code>	Criar e iniciar containers em background
<code>docker compose up --build</code>	Rebuild + iniciar
<code>docker compose down</code>	Parar e remover containers + redes
<code>docker compose down -v</code>	Down + remover volumes
<code>docker compose down --rm all</code>	Down + remover imagens
<code>docker compose ps</code>	Listar status dos containers
<code>docker compose logs -f</code>	Logs em tempo real
<code>docker compose stop</code>	Parar sem remover
<code>docker compose start</code>	Iniciar containers parados
<code>docker compose restart</code>	Reiniciar serviços
<code>docker compose exec <svc> sh</code>	Shell em container ativo
<code>docker compose run --rm <svc> cmd</code>	Comando único em container temporário
<code>docker compose up --scale api=3</code>	Escalar réplicas
<code>docker compose build</code>	Build das imagens
<code>docker compose pull</code>	Baixar imagens atualizadas
<code>docker compose config</code>	Validar e exibir compose final



Otimização, Segurança e Boas Práticas

Construa imagens menores, mais seguras e eficientes. Domine multi-stage builds, hardening de containers e monitoramento de recursos.

Multi-stage Build

Conceito

OTIMIZAÇÃO

Multi-stage build usa múltiplos FROM no mesmo Dockerfile. Cada estágio pode copiar artefatos do anterior, resultando em imagens finais muito menores — sem ferramentas de build, sem código-fonte, apenas o binário/artefato necessário.

```
# Benefícios:  
# ✓ Imagem final contém APENAS o necessário para execução  
# ✓ Ferramentas de build ficam no estágio intermediário  
# ✓ Redução drástica de tamanho (ex: Go 1.2GB → 10MB)  
# ✓ Menor superfície de ataque  
# ✓ Um único Dockerfile para build + runtime
```

Node.js Multi-stage EXEMPLO

Build com dependências de dev, runtime apenas com produção.

```
# Estágio 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Estágio 2: Runtime
FROM node:20-alpine AS runtime
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY --from=builder /app/dist ./dist
EXPOSE 3000
USER node
CMD ["node", "dist/server.js"]
```

Go com scratch (~10MB) EXEMPLO

Compilação estática em Go resulta em imagem mínima com scratch.

```
# Estágio 1: Compilação
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux \
    go build -ldflags="-s -w" \
    -o /app/server ./cmd/main.go

# Estágio 2: Imagem mínima
FROM scratch
COPY --from=builder /app/server /server
COPY --from=builder /etc/ssl/certs/ca-certificates.crt \
    /etc/ssl/certs/
EXPOSE 8080
ENTRYPOINT ["/server"]
# Resultado: ~10MB!
```

React + Nginx**EXEMPLO**

Build do React no Node, serve estático com Nginx.

```
# Estágio 1: Build React
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Estágio 2: Nginx serve os estáticos
FROM nginx:alpine
COPY --from=builder /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

# Resultado: ~25MB
```

Otimização de Cache e Layers

Ordem das Layers CACHE

A ordem importa! Coloque o que muda menos primeiro para maximizar cache hits.

```
# ❌ RUIM – qualquer mudança no código
# invalida cache do npm install
COPY . .
RUN npm install

# ✅ BOM – package.json muda raramente
# npm install usa cache na maioria dos builds
COPY package*.json ./
RUN npm ci
COPY . .
```

Combine RUN + Alpine CACHE

Reduza layers combinando comandos e use imagens Alpine.

```
# ❌ RUIM – 3 layers, cache de apt desnecessário
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# ✅ BOM – 1 layer, limpa cache
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        curl ca-certificates && \
    rm -rf /var/lib/apt/lists/*

# ✅ MELHOR – use Alpine (5MB base)
FROM node:20-alpine
# Alpine usa apk ao invés de apt
```

.dockerignore **OTIMIZAÇÃO**

Exclua arquivos desnecessários do build context para builds mais rápidos.

```
# .dockerignore
node_modules
npm-debug.log
.git
.gitignore
.env
.env.*
Dockerfile
docker-compose*.yaml
.dockerignore
README.md
docs/
tests/
coverage/
.vscode/
*.md
```

Comparação de Tamanhos **REFERÊNCIA**

Escolha a imagem base adequada para seu caso de uso.

```
# Tamanhos aproximados de imagens base:
# node:20          → ~1.1GB
# node:20-slim     → ~200MB
# node:20-alpine   → ~130MB
# golang:1.22      → ~800MB
# golang:1.22-alpine → ~250MB
# scratch          → 0MB (vazio)
# alpine           → ~5MB
# ubuntu:24.04     → ~78MB
# debian:bookworm-slim → ~75MB
# nginx:alpine     → ~40MB
```


Segurança de Containers

Usuário não-root

SEGURANÇA

Nunca execute containers como root em produção. Crie um usuário dedicado.

```
# Criar grupo e usuário sem privilégios
RUN addgroup -S appgroup && \
    adduser -S appuser -G appgroup

# Copiar arquivos com ownership correto
COPY --chown=appuser:appgroup . /app

# Mudar para usuário não-root
USER appuser

# Verificar: docker exec container whoami
# Resultado: appuser (não root!)
```

Vulnerability Scanning

SEGURANÇA

Escaneie imagens para vulnerabilidades conhecidas antes de deploy.

```
# Docker Scout (integrado ao Docker Desktop)
docker scout cves minha-imagem:latest
docker scout recommendations minha-imagem:latest

# Trivy (open source, CI/CD friendly)
trivy image minha-imagem:latest
trivy image --severity HIGH,CRITICAL app:v1
trivy image --exit-code 1 app:v1 # Falha no CI

# Snyk
docker scan minha-imagem:latest
```

Boas Práticas de Segurança

SEGURANÇA

Checklist de segurança para imagens e containers em produção.

```
# ✓ Use tags específicas (NUNCA :latest)
FROM node:20.11.0-alpine

# ✓ Não coloque secrets na imagem
# Use env vars ou secrets manager

# ✓ Filesystem read-only
docker run --read-only app:v1

# ✓ Drop ALL capabilities
docker run --cap-drop ALL app:v1

# ✓ Nunca monte o Docker socket
# ✗ -v /var/run/docker.sock:/var/run/docker.sock

# ✓ Docker Content Trust (assinatura)
export DOCKER_CONTENT_TRUST=1
docker pull nginx # Verifica assinatura
```

Runtime Hardening

SEGURANÇA

Restrições adicionais ao executar containers.

```
# Combinação completa de segurança
docker run -d \
  --name app-secure \
  --read-only \
  --cap-drop ALL \
  --cap-add NET_BIND_SERVICE \
  --security-opt no-new-privileges \
  --tmpfs /tmp:rw,noexec,nosuid \
  --memory 256m \
  --cpus 0.5 \
  --pids-limit 100 \
  --user 1000:1000 \
  app:v1
```

Monitoramento e Debug

docker stats

MONITORAR

Monitore CPU, memória, rede e I/O em tempo real.

```
# Tempo real (atualiza a cada segundo)
docker stats

# Snapshot único (para scripts)
docker stats --no-stream

# Container específico
docker stats meu-app

# Formato customizado
docker stats --format \
  "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

`docker system df` **DISCO**

Verifique uso de disco por tipo de objeto Docker.

```
# Resumo de uso de disco
docker system df

# Detalhado por objeto
docker system df -v

# Saída exemplo:
# TYPE          TOTAL    ACTIVE  SIZE    RECLAIMABLE
# Images        15       5        4.2GB   2.8GB (66%)
# Containers    8        3        120MB   85MB (70%)
# Volumes       12       4        1.5GB   900MB (60%)
# Build Cache   -        -        2.1GB   2.1GB
```

`docker system prune` **LIMPEZA**

Limpe recursos não utilizados para liberar espaço.

```
# Limpar containers parados + imagens dangling
docker system prune

# Limpar TUDO não utilizado (cuidado!)
docker system prune -a

# Incluir volumes (dados persistentes!)
docker system prune -a --volumes

# Sem confirmação (scripts)
docker system prune -af
```

`docker events / top / diff` DEBUG

Ferramentas avançadas de debug e inspeção.

```
# Eventos em tempo real do daemon
docker events
docker events --filter type=container

# Processos dentro do container
docker top meu-app
docker top meu-app aux

# Mudanças no filesystem do container
docker diff meu-app
# A = adicionado, C = modificado, D = deletado

# Salvar estado atual como nova imagem
docker commit meu-app debug-snapshot:v1

# Exportar filesystem como tar
docker export meu-app > app-fs.tar
```

Resource Limits

RECURSOS

Limite recursos para evitar que um container consuma toda a máquina.

```
# Limite de memória (container é killed se exceder)
docker run --memory 512m --memory-swap 1g app:v1

# Limite de CPU
docker run --cpus 1.5 app:v1           # 1.5 cores
docker run --cpu-shares 512 app:v1     # Peso relativo

# Limite de I/O de disco
docker run --device-read-bps /dev/sda:10mb app:v1
docker run --device-write-bps /dev/sda:5mb app:v1

# Verificar limites aplicados
docker inspect --format '{{.HostConfig.Memory}}' meu-app
docker stats --no-stream meu-app
```



Docker em Produção e CI/CD

Leve seus containers para produção com registries privados, pipelines CI/CD automatizados, Docker Swarm e estratégias de troubleshooting.

Registry Privado

`docker login / push / pull` **REGISTRY**

Autentique e publique imagens em registries privados (ECR, ACR, GCR, Docker Hub).

```
# Login no Docker Hub
docker login

# Login em registry privado (AWS ECR)
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
    123456789.dkr.ecr.us-east-1.amazonaws.com

# Tag para o registry
docker tag app:v1 \
  123456789.dkr.ecr.us-east-1.amazonaws.com/app:v1

# Push para o registry
docker push \
  123456789.dkr.ecr.us-east-1.amazonaws.com/app:v1

# Pull do registry
docker pull \
  123456789.dkr.ecr.us-east-1.amazonaws.com/app:v1
```

Estratégias de Tagging

VERSIONAMENTO

Use tags semânticas para rastreabilidade e rollback.

```
# Semantic Versioning (recomendado)
docker tag app:latest registry/app:1.2.3
docker tag app:latest registry/app:1.2
docker tag app:latest registry/app:1

# Git SHA (rastreabilidade exata)
GIT_SHA=$(git rev-parse --short HEAD)
docker tag app:latest registry/app:${GIT_SHA}

# Data + Build number
docker tag app:latest registry/app:2024.01.15-42

# Combinação (melhor prática)
docker tag app:latest registry/app:1.2.3
docker tag app:latest registry/app:1.2.3-abc1234
docker tag app:latest registry/app:latest

# ⚠ NUNCA use apenas :latest em produção!
```

Docker Compose para Produção

docker-compose.prod.yml

PRODUÇÃO

Compose otimizado para produção com imagens do registry, healthchecks e limites de recursos.

```
services:
  api:
    image: 123456789.dkr.ecr.us-east-1.amazonaws.com/app:1.2.3
    restart: unless-stopped
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
        reservations:
          cpus: "0.25"
          memory: 128M
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:4000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"
    environment:
      - NODE_ENV=production
    ports:
      - "4000:4000"

  db:
    image: postgres:16-alpine
    restart: always
    volumes:
      - pgdata:/var/lib/postgresql/data
    deploy:
      resources:
        limits:
          memory: 1G
    logging:
      driver: json-file
      options:
```

```
    max-size: "20m"
    max-file: "10"

volumes:
  pgdata:
```

Logging em Produção

Logging Drivers

OBSERVABILIDADE

Configure drivers de log para centralizar e gerenciar logs em produção.

```
# json-file (padrão) – com rotação
docker run -d \
  --log-driver json-file \
  --log-opt max-size=10m \
  --log-opt max-file=5 \
  app:v1

# syslog – enviar para servidor syslog
docker run -d \
  --log-driver syslog \
  --log-opt syslog-address=tcp://logserver:514 \
  --log-opt tag="{{.Name}}" \
  app:v1

# fluentd – agregação com Fluentd/Fluent Bit
docker run -d \
  --log-driver fluentd \
  --log-opt fluentd-address=localhost:24224 \
  --log-opt tag="docker.{{.Name}}" \
  app:v1

# awslogs – direto para CloudWatch
docker run -d \
  --log-driver awslogs \
  --log-opt awslogs-region=us-east-1 \
  --log-opt awslogs-group=/ecs/app \
  --log-opt awslogs-stream-prefix=prod \
  app:v1
```

CI/CD com GitHub Actions

[.github/workflows/docker.yml](#) CI/CD

Pipeline completo: build, scan, push para registry com cache otimizado.

```

name: Docker Build & Push

on:
  push:
    branches: [main]
    tags: ["v*"]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Extract metadata (tags, labels)
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
          tags: |

```

```
    type=semver,pattern={{version}}
    type=semver,pattern={{major}}.{{minor}}
    type=sha,prefix=
    type=ref,event=branch

- name: Build and Push
  uses: docker/build-push-action@v5
  with:
    context: .
    push: ${{{ github.event_name != 'pull_request' }}}
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

CI/CD com GitLab CI

`.gitlab-ci.yml` **CI/CD**

Pipeline GitLab com build, test e deploy para registry.


```

stages:
  - build
  - test
  - deploy

variables:
  IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
  IMAGE_LATEST: $CI_REGISTRY_IMAGE:latest

build:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - docker build --cache-from $IMAGE_LATEST -t $IMAGE_TAG -t $IMAGE_LATEST .
    - docker push $IMAGE_TAG
    - docker push $IMAGE_LATEST
  only:
    - main
    - tags

test:
  stage: test
  image: $IMAGE_TAG
  script:
    - npm run test
  needs: [build]

scan:
  stage: test
  image: aquasec/trivy:latest
  script:
    - trivy image --exit-code 1 --severity HIGH,CRITICAL $IMAGE_TAG
  needs: [build]
  allow_failure: true

deploy:

```

```
stage: deploy
image: docker:24
services:
  - docker:24-dind
script:
  - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  - docker pull $IMAGE_TAG
  - docker tag $IMAGE_TAG $CI_REGISTRY_IMAGE:production
  - docker push $CI_REGISTRY_IMAGE:production
only:
  - tags
when: manual
```

Docker Swarm

Inicialização e Nodes CLUSTER

Configure um cluster Swarm para orquestração nativa do Docker.

```
# Inicializar Swarm (no manager)
docker swarm init --advertise-addr 192.168.1.10

# Token para workers se juntarem
docker swarm join-token worker

# Listar nodes do cluster
docker node ls

# Promover worker a manager
docker node promote worker-01

# Remover node do cluster
docker node rm worker-02
```

Services

DEPLOY

Crie e gerencie serviços distribuídos no cluster.

```
# Criar serviço com réplicas
docker service create \
  --name api \
  --replicas 3 \
  --publish 8080:4000 \
  --env NODE_ENV=production \
  registry/app:1.2.3

# Listar serviços
docker service ls

# Ver tasks (containers) do serviço
docker service ps api

# Escalar réplicas
docker service scale api=5

# Logs do serviço
docker service logs -f api
```

Updates e Rollback

DEPLOY

Rolling updates e rollback automático em caso de falha.

```
# Update da imagem (rolling update)
docker service update \
  --image registry/app:1.3.0 \
  --update-parallelism 1 \
  --update-delay 10s \
  --update-failure-action rollback \
  api

# Rollback manual
docker service rollback api

# Remover serviço
docker service rm api
```

Stacks

DEPLOY

Deploy de aplicações completas via compose file no Swarm.

```
# Deploy stack a partir de compose file
docker stack deploy -c docker-compose.yml myapp

# Listar stacks
docker stack ls

# Serviços da stack
docker stack services myapp

# Tasks da stack
docker stack ps myapp

# Remover stack
docker stack rm myapp
```

Cleanup e Manutenção

Comandos de Limpeza

MANUTENÇÃO

Mantenha o ambiente limpo removendo recursos não utilizados.

```
# Ver uso de disco
docker system df

# Limpar containers parados
docker container prune

# Limpar imagens sem tag (dangling)
docker image prune

# Limpar imagens não utilizadas por nenhum container
docker image prune -a

# Limpar volumes órfãos
docker volume prune

# Limpar redes não utilizadas
docker network prune

# Limpar build cache
docker builder prune

# NUCLEAR: limpar TUDO não utilizado + volumes
docker system prune -a --volumes

# ⚠ Cuidado: remove dados persistentes!
```

Troubleshooting

Container Crashando

DEBUG

Diagnostique containers que reiniciam ou falham ao iniciar.

```
# Ver logs do container que crashou
docker logs --tail 100 meu-app
docker logs --since 5m meu-app

# Inspecionar exit code
docker inspect meu-app --format='{{.State.ExitCode}}'

# ExitCode 137 = OOM Killed (falta memória)
# ExitCode 1   = Erro da aplicação
# ExitCode 0   = Saiu normalmente

# Ver estado detalhado
docker inspect meu-app --format='{{json .State}}' | jq

# Eventos do daemon
docker events --filter container=meu-app

# Ver se foi OOM killed
docker inspect meu-app --format='{{.State.OOMKilled}}'
```

Debug de Rede

DEBUG

Diagnostique problemas de conectividade entre containers.

```
# Container de debug na mesma rede
docker run --rm -it --network app_default \
  nicolaka/netshoot sh

# Testar DNS interno
nslookup api
nslookup db

# Testar conectividade
ping api
curl -v http://api:4000/health

# Inspeccionar rede
docker network inspect app_default

# Ver portas mapeadas
docker port meu-app
```

Health Status

DEBUG

Verifique o status de saúde dos containers.

```
# Ver health status
docker inspect meu-app \
  --format='{{.State.Health.Status}}'
# healthy | unhealthy | starting

# Histórico de health checks
docker inspect meu-app \
  --format='{{json .State.Health}}' | jq

# Filtrar containers unhealthy
docker ps --filter health=unhealthy

# Ver logs do healthcheck
docker inspect meu-app \
  --format='{{range .State.Health.Log}}{{.Output}}{{end}}'
```


Inspect com jq DEBUG

Use jq para extrair informações específicas do inspect.

```
# IP do container
docker inspect meu-app | \
  jq -r '.[0].NetworkSettings.IPAddress'
```

```
# Variáveis de ambiente
docker inspect meu-app | \
  jq '.[0].Config.Env'
```

```
# Mounts/Volumes
docker inspect meu-app | \
  jq '.[0].Mounts'
```

```
# Port bindings
docker inspect meu-app | \
  jq '.[0].NetworkSettings.Ports'
```

```
# Restart count
docker inspect meu-app | \
  jq '.[0].RestartCount'
```

Resumo: Checklist de Produção

Área	Prática
Imagem	Multi-stage build, Alpine, tag específica (nunca :latest)
Segurança	Non-root user, --read-only, --cap-drop ALL, scan de vulnerabilidades
Recursos	Limites de CPU e memória definidos (deploy.resources.limits)
Healthcheck	Endpoint /health, interval 30s, retries 3
Logging	Driver json-file com max-size e max-file configurados
Restart	restart: unless-stopped para serviços críticos
Registry	Imagens versionadas no registry privado (ECR/GCR/ACR)
CI/CD	Build automatizado, scan, push e deploy via pipeline
Backup	Volumes com backup regular, named volumes para dados
Cleanup	Prune agendado, monitorar docker system df